

```

# Step 1: Install plotly if it's not already available (Colab usually has it pre-installed)
!pip install plotly -q

import plotly.graph_objects as go
import pandas as pd

# Step 2: Define your source categories (Left side) and target categories (Right side)
# We will create a unique list of nodes.
# Index 0-11 are the Sectors, Index 12-14 are the Recipients.
nodes = [
    # Left side nodes (Sectors)
    "Lawyers & Lobbyists", "Other", "Defense", "Communic/Electronics",
    "Transportation", "Construction", "Finance/Insur/RealEst", "Health",
    "Ideology/Single-Issue", "Agribusiness", "Misc Business", "Energy/Nat Resources",
    # Right side nodes (Recipients)
    "Democrats", "Republicans", "Outside Spending Grps"
]

# Step 3: Define the flows (source index -> target index -> value/amount)
# Node indices map directly to the 'nodes' list above (e.g., 0 = Lawyers & Lobbyists, 12 = Democrats)
data = [
    # Source, Target, Value
    [0, 12, 50], [0, 13, 20], [0, 14, 5], # Lawyers & Lobbyists
    [1, 12, 45], [1, 13, 25], [1, 14, 10], # Other
    [2, 12, 15], [2, 13, 35], [2, 14, 8], # Defense
    [3, 12, 30], [3, 13, 15], [3, 14, 12], # Communic/Electronics
    [4, 12, 12], [4, 13, 22], [4, 14, 4], # Transportation
    [5, 12, 10], [5, 13, 28], [5, 14, 6], # Construction
    [6, 12, 60], [6, 13, 75], [6, 14, 30], # Finance/Insur/RealEst
    [7, 12, 40], [7, 13, 30], [7, 14, 5], # Health
    [8, 12, 55], [8, 13, 40], [8, 14, 80], # Ideology/Single-Issue
    [9, 12, 8], [9, 13, 24], [9, 14, 3], # Agribusiness
    [10, 12, 20], [10, 13, 30], [10, 14, 10], # Misc Business
    [11, 12, 12], [11, 13, 48], [11, 14, 15] # Energy/Nat Resources
]

# Convert the data list into a cleaner DataFrame structure
df = pd.DataFrame(data, columns=['source', 'target', 'value'])

# Step 4: Define custom colors for the target nodes to match your image
# Democrats = Blue, Republicans = Red, Outside Spending = Yellow/Gold
# Left nodes can remain a neutral grey or inherit transparency
node_colors = ["#7f7f7f"] * 12 + ["#3366cc", "#ff6666", "#ffcc00"]

# Assign ribbon colors based on their target (where they flow to)
def get_link_color(target_index):
    if target_index == 12: return "rgba(51, 102, 204, 0.4)" # Translucent Blue
    elif target_index == 13: return "rgba(255, 102, 102, 0.4)" # Translucent Red
    else: return "rgba(255, 204, 0, 0.4)" # Translucent Yellow

df['link_color'] = df['target'].apply(get_link_color)

# Step 5: Construct the Sankey Diagram
fig = go.Figure(data=[go.Sankey(
    node = dict(
        pad = 15,
        thickness = 20,
        line = dict(color = "black", width = 0.5),
        label = nodes,
        color = node_colors
    ),
    link = dict(
        source = df['source'].tolist(),
        target = df['target'].tolist(),
        value = df['value'].tolist(),
        color = df['link_color'].tolist()
    )
)]) # Removed the extra ']' here

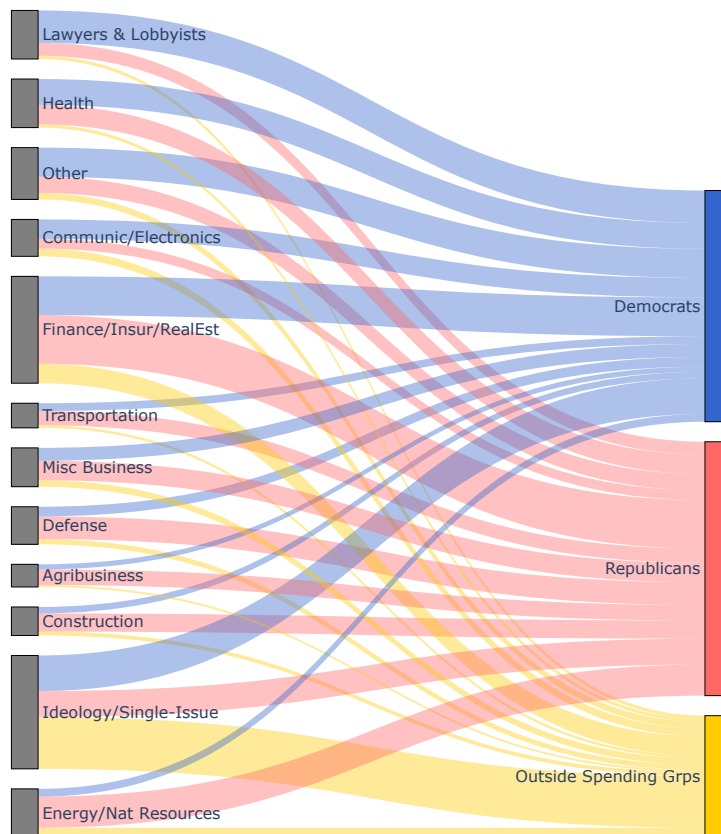
# Step 6: Customize layout text, font, and dimensions
fig.update_layout(
    title_text="Contributions During 2018 Election Cycle<br><sub>(by Sector)</sub>",
    title_x=0.5, # Center the title
    font_size=12,
    width=700,
    height=800,
    plot_bgcolor='white',
    paper_bgcolor='white'
)

# Step 7: Display the interactive graph inside Google Colab
fig.show()

```

Contributions During 2018 Election Cycle

(by Sector)



```
# Step 1: Install plotly if needed
!pip install plotly -q

import plotly.express as px
import pandas as pd

# Step 2: Re-create the complete flow dataset from the first image
# Mapping every sector contribution to its respective recipient group
data = {
    "Total": ["2018 Election Cycle"] * 36,
    "Recipient": [
        "Democrats", "Democrats", "Democrats", "Democrats", "Democrats", "Democrats",
        "Democrats", "Democrats", "Democrats", "Democrats", "Democrats", "Democrats",
        "Republicans", "Republicans", "Republicans", "Republicans", "Republicans", "Republicans",
        "Republicans", "Republicans", "Republicans", "Republicans", "Republicans", "Republicans",
        "Outside Spending Grps", "Outside Spending Grps", "Outside Spending Grps", "Outside Spending Grps",
        "Outside Spending Grps", "Outside Spending Grps", "Outside Spending Grps", "Outside Spending Grps",
        "Outside Spending Grps", "Outside Spending Grps", "Outside Spending Grps", "Outside Spending Grps"
    ],
    "Sector": [
        "Lawyers & Lobbyists", "Other", "Defense", "Communic/Electronics", "Transportation", "Construction", "Finance/Insur/",
        "Lawyers & Lobbyists", "Other", "Defense", "Communic/Electronics", "Transportation", "Construction", "Finance/Insur/",
        "Lawyers & Lobbyists", "Other", "Defense", "Communic/Electronics", "Transportation", "Construction", "Finance/Insur/"
    ],
    "Amount (Millions)": [
        50, 45, 15, 30, 12, 10, 60, 40, 55, 8, 20, 12, # Democrats
        20, 25, 35, 15, 22, 28, 75, 30, 40, 24, 30, 48, # Republicans
        5, 10, 8, 12, 4, 6, 30, 5, 80, 3, 10, 15 # Outside Spending
    ]
}

df = pd.DataFrame(data)

# Step 3: Set up specific color mappings to resemble the original color choices
color_map = {
    "2018 Election Cycle": "#d3d3d3", # Grey root
    "Democrats": "#3366cc", # Blue
    "Republicans": "#cc3333", # Red
    "Outside Spending Grps": "#cc9933", # Yellow
}
```

```
Democrat = "#3399cc", # Blue
"Republicans": "#ff6666", # Red
"Outside Spending Grps": "#ffcc00" # Yellow/Gold
}

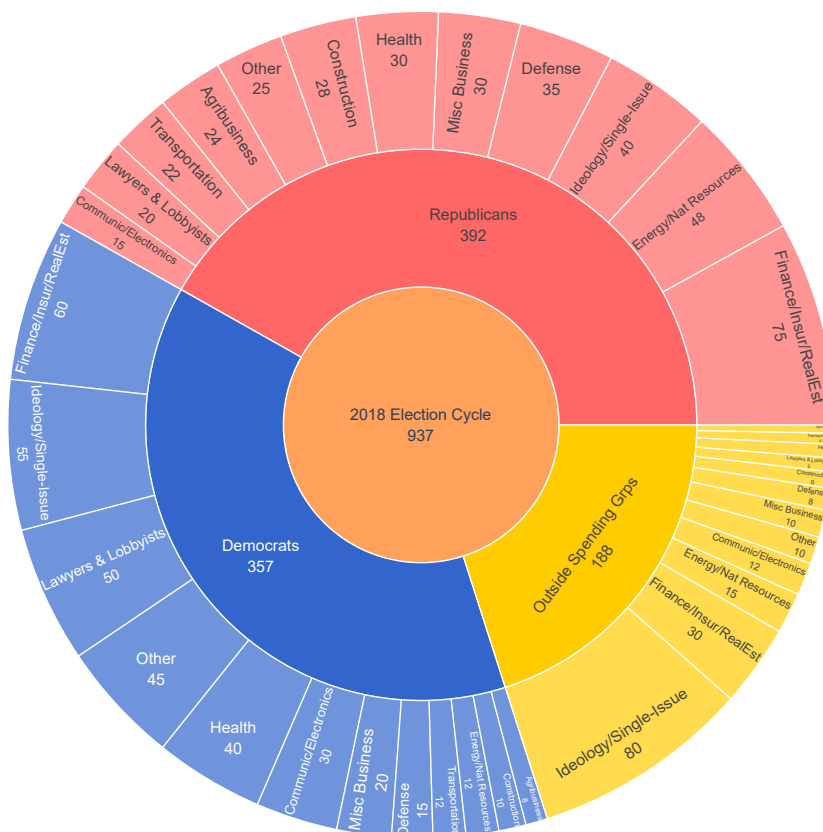
# Step 4: Create the Sunburst Chart
fig = px.sunburst(
    df,
    path=["Total", "Recipient", "Sector"], # Moving from the inside ring outwards
    values="Amount (Millions)",
    color="Recipient", # Base sector colors on the recipient node
    color_discrete_map=color_map,
    title="Contributions During 2018 Election Cycle (Hierarchical Structure)"
)

# Step 5: Clean up layout text styling and sizing
fig.update_layout(
    title_x=0.5,
    width=800,
    height=800,
    font_family="Arial"
)

# Optional: Adjust trace to display both labels and values cleanly inside the rings
fig.update_traces(
    textinfo="label+value"
)

# Step 6: Render chart inside Google Colab
fig.show()
```

Contributions During 2018 Election Cycle (Hierarchical Structure)



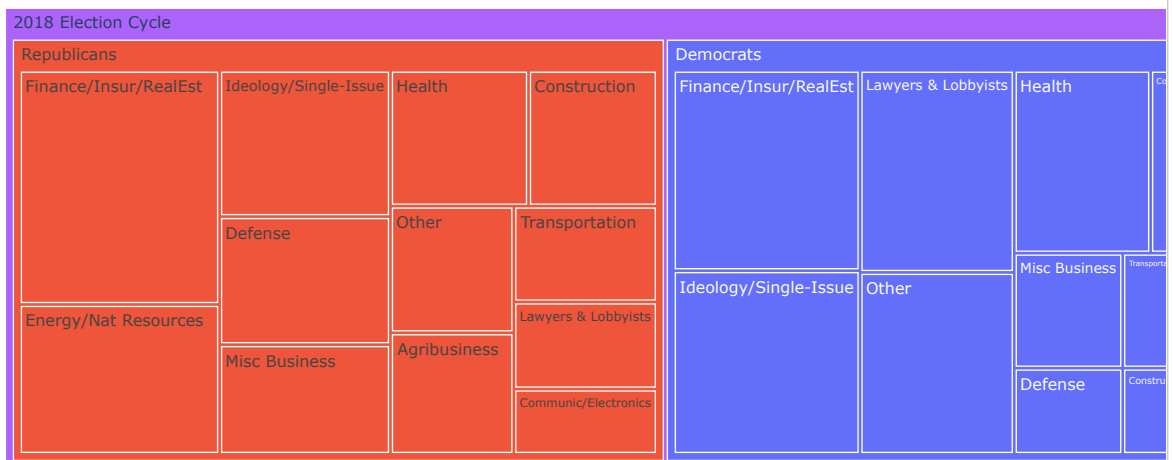
```
fig = px.sunburst(
    df,
    path=["Total", "Recipient", "Sector"],
    values="Amount (Millions)",
    color="Recipient",
    title="Election Contributions Sunburst"
)
fig.show()
```

Election Contributions Sunburst



```
fig = px.treemap(
    df,
    path=["Total", "Recipient", "Sector"],
    values="Amount (Millions)",
    color="Recipient",
    title="Election Contributions Treemap"
)
fig.show()
```

Election Contributions Treemap



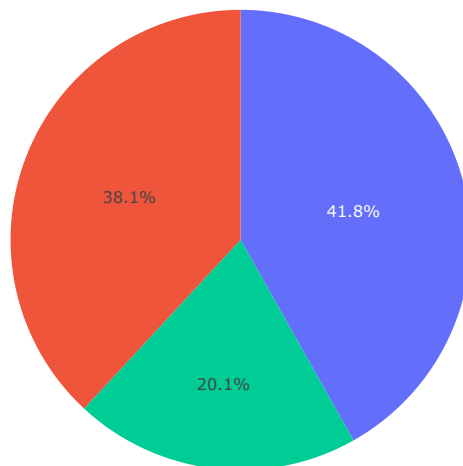
```
import plotly.express as px

recipient_df = df.groupby("Recipient")["Amount (Millions)"].sum().reset_index()

fig = px.pie(
```

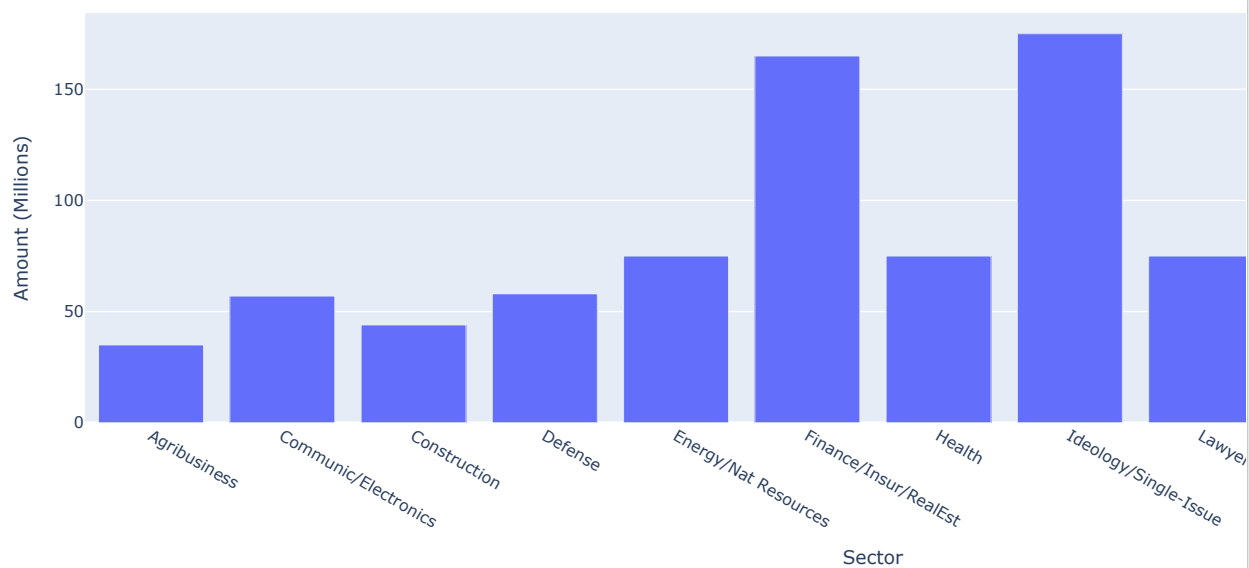
```
recipient_df,\nnames="Recipient",\nvalues="Amount (Millions)",\ntitle="Funding by Recipient"\n)\n\nfig.show()
```

Funding by Recipient



```
sector_df = df.groupby("Sector")["Amount (Millions)"].sum().reset_index()\n\nfig = px.bar(\n    sector_df,\n    x="Sector",\n    y="Amount (Millions)",\n    title="Sector-wise Contributions"\n)\n\nfig.show()
```

Sector-wise Contributions

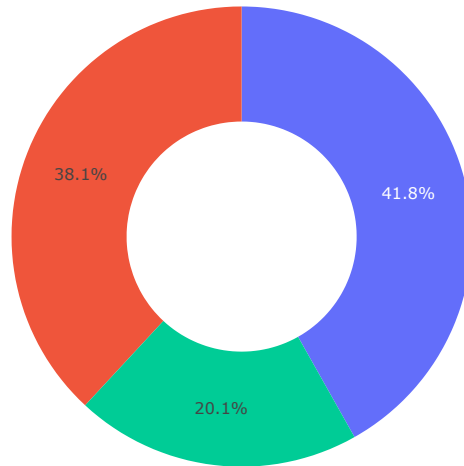


```
recipient_df = df.groupby("Recipient")["Amount (Millions)"].sum().reset_index()\n\nfig = px.pie(\n    recipient_df,\n    names="Recipient",\n
```

```
values="Amount (Millions)",
hole=0.5,
title="Recipient Contribution Share"
)

fig.show()
```

Recipient Contribution Share



```
recipient_df = df.groupby("Recipient")["Amount (Millions)"].sum().reset_index()

fig = px.funnel(
    recipient_df,
    x="Amount (Millions)",
    y="Recipient",
    title="Funding Funnel"
)

fig.show()
```

Funding Funnel

